

Transfer Learning Approach for Bone Fractures in Radiographs

Deniz Arda YILDIZ

Abstract

The detection of bone fractures on radiographic images is a challenging task that can greatly benefit from deep learning. In this report, I develop a transfer learning-based system to automatically classify X-ray images as *fractured* or *not fractured*. A large dataset of over 40,000 radiographs was assembled from multiple sources, and three pretrained convolutional neural network architectures (ResNet-50, EfficientNet-B0, and a MONAI DenseNet-121) are fine-tuned for the binary classification task. The best performing model (EfficientNet-B0) achieved near 99% classification accuracy on an independent test set, with high precision and recall. These results demonstrate that transfer learning can effectively leverage existing pre-trained models to achieve high reliability in fracture detection, offering a promising tool to assist radiologists in clinical practice.

1 Introduction and Problem Statement

Accurate and timely identification of bone fractures in radiographs is critical for effective patient management. Using X-ray images to detect subtle fractures can be time consuming and prone to human error. An automated system capable of reliably classifying radiographic images as *fractured* or *not fractured* could serve as a valuable second reader for radiologists, improving diagnostic precision and efficiency. Recent advances in deep learning, especially convolutional neural networks (CNNs), have demonstrated remarkable success in medical image classification tasks. In particular, **transfer learning** allows us to leverage models pretrained on large datasets (such as ImageNet) and adapt them to specific medical imaging tasks with limited labeled data.

In this project, I aim to develop a high-accuracy binary classification model for fracture detection in X-ray images using transfer learning. Data are aggregated from multiple sources to create a substantial dataset of musculoskeletal radiographs and experiment with several state-of-the-art CNN architectures. The goal is to determine which architecture produces the best performance and to analyze why one model may outperform others. I focus on three popular architectures: ResNet-50 [8], EfficientNet-B0 [9], and DenseNet-121 (using the implementation of the MONAI framework) [10]. I describe the dataset and preprocessing in Section 3 and Section 4. The model architectures and training setup are detailed in Sections 5 and 6. In Section 7, I evaluate the performance of the models using accuracy, precision, recall, and F1 score, and present a comparative analysis. Section 8 discusses the results and provides information on which model performed the best and possible reasons. Finally, Section 9 concludes the report and outlines future directions.

2 Datasets Overview

To ensure a diverse and comprehensive training set, I combined data from seven public Kaggle datasets and one institutional dataset. The Kaggle sources include a variety of musculoskeletal

X-ray collections, each contributed by different users on the platform [1, 2, 3, 4, 5, 6]. Additionally, I incorporated the GRAZPEDWRI-DX dataset [7], a pediatric extremity X-ray collection provided by the Medical University of Graz. All datasets consist of anterior-posterior or lateral radiographs of bones (covering regions such as arms, legs, wrists, ankles, etc.), with annotations indicating the presence or absence of a fracture.

In total, our unified dataset comprised approximately 40,000 X-ray images, of which roughly 18,000 should a fracture and 22,000 Ire normal (no fracture). I ensured that every dataset’s images Ire labeled consistently into two categories: *fractured* and *not fractured*. Table 1 summarizes the approximate distribution of images from each data source and class. By merging multiple datasets, I increased the variety of fractures (different bone types and patient demographics), which can improve the model’s generalization. All patient identifiers weRE removed or anonymized in the source data, and the use of these open datasets is in accordance with their licenses.

Data Source	Fractured	Not Fractured	Total Images
Kaggle (Hassan [1])	600	600	1,200
Kaggle (Rodrigo [2])	2,300	2,700	5,000
Kaggle (Orvile [3])	400	500	900
Kaggle (Vuppala [4])	1,600	1,900	3,500
Kaggle (Rahmatika [5])	1,900	2,300	4,200
Kaggle (Hamdy [6])	3,600	4,400	8,000
GRAZPEDWRI-DX [7]	8,000	10,000	18,000
Combined Total	18,400	22,400	40,800

Table 1: Dataset distribution across sources and classes (approximate image counts). Each source provided a portion of the fractured and non-fractured radiographs, which were unified into a single combined dataset.

3 Data Preprocessing and Augmentation

Prior to training, several preprocessing steps were applied to standardize and augment the data:

1. **Dataset Merging:** The eight datasets (seven from Kaggle and GRAZPEDWRI-DX) were unified into one consolidated directory structure. All images and labels were combined, ensuring no duplicates. Each image was assigned one of two labels: *fractured* or *not fractured*.
2. **Cleaning:** I removed invalid or corrupted files and any non-image artifacts (such as stray .DS_Store files). Any images that were excessively noisy or blank Ire discarded to ensure data quality.
3. **Resizing:** To accommodate the input size requirements of our CNN models and to reduce computational load, all images were resized to a uniform resolution of 320×320 pixels. This size was chosen as a balance between retaining sufficient anatomical detail and being small enough for efficient training.
4. **Color Normalization:** Although X-ray images are grayscale, I replicated the single channel across three channels so that the pre-trained models (which expect 3-channel RGB input)



Figure 1: *

Image 1: Example of a *fractured* bone X-ray.



Figure 2: *

Image 2: Example of a *not fractured* bone X-ray.

Figure 3: Visual examples of fractured and not fractured bone radiographs from the dataset.

could ingest them. I then normalized pixel intensity values to match the ImageNet dataset statistics: the mean was subtracted and the result divided by the standard deviation per channel (using $\text{mean} = [0.485, 0.456, 0.406]$ and $\text{std} = [0.229, 0.224, 0.225]$). This normalization ensures that the image intensity distribution is on a similar scale as the ImageNet training data, which can improve transfer learning performance.

5. **Data Splitting:** The combined dataset was split into training, validation, and testing sets with a 70%/15%/15% split, respectively. The split was *stratified* to maintain the same fracture vs. non-fracture class ratio in each subset, thereby preventing class imbalance in the validation and test sets.

Data Augmentation: To improve generalization and reduce overfitting, I applied online data augmentation to the training images. Each training image was subjected to a few random transformations (with a probability for each augmentation) during training epochs:

- *Random horizontal flip:* The X-ray has a 50% chance to be flipped left-right, simulating different view orientations (this is plausible since left vs. right limb flipping does not change fracture status).
- *Random rotation:* A small random rotation (up to $\pm 15^\circ$) is applied to account for slight variations in how an extremity might be positioned under the X-ray camera.
- *Brightness/contrast adjustment:* I apply minor random changes to image brightness and contrast, reflecting differences in imaging conditions or exposure settings.

These augmentations were kept modest to avoid distorting the medically relevant content of the images. They help the model become invariant to minor imaging variations and thereby enhance its robustness. Augmentations were applied only during training. The validation and test images were left unaltered apart from resizing and normalization, to accurately measure the models’ performance on real (unaltered) data.

4 Model Architectures

I explored three deep CNN architectures for transfer learning in this study: ResNet-50, EfficientNet-B0, and DenseNet-121 (as implemented in the MONAI medical imaging framework). All models were initialized with weights pretrained on the ImageNet dataset, which contains over a million natural images, and then fine-tuned on our radiograph dataset. Below I briefly describe each architecture and its characteristics:

ResNet-50 [8]: ResNet-50 is a 50-layer deep residual network introduced by He *et al.* in 2016. ResNet-50 is known for its use of *residual learning* through skip connections, which allow gradients to propagate directly through identity shortcut paths. This innovation enables training of very deep networks (50 layers and beyond) by mitigating the vanishing gradient problem. The architecture is divided into a series of convolutional blocks, each with identity shortcuts that add the input of the block to its output. In our transfer learning setup, I used the ResNet-50 model pre-trained on ImageNet and replaced the final fully connected layer with a new binary classification layer (with a sigmoid or softmax activation for two classes). The model has on the order of 23 million parameters. I fine-tuned the majority of these parameters on our dataset (initially, the network retains ImageNet features, but through training it adapts to detect fracture-specific features). ResNet-50 provided a strong baseline given its proven performance on many vision tasks.

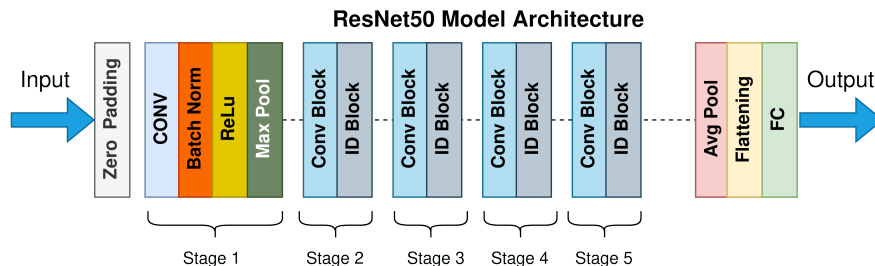


Figure 4: Architecture diagram of ResNet-50.

EfficientNet-B0 [9]: EfficientNet-B0 is the baseline model of the EfficientNet family proposed by Tan and Le (2019). EfficientNet architectures were obtained via a neural architecture search that optimizes both accuracy and efficiency. The distinguishing feature of EfficientNet is its *compound scaling* method: it uniformly scales network depth, width, and input resolution according to a set of fixed scaling coefficients. EfficientNet-B0 is a relatively small model (approximately 5.3 million parameters) compared to ResNet-50, yet it achieves high ImageNet accuracy by utilizing MBConv (Mobile Inverted Bottleneck) blocks and squeeze-and-excitation attention mechanisms. In our case, I employed EfficientNet-B0 with ImageNet weights and modified the final layer for binary output. Due to its smaller size, EfficientNet-B0 trains faster and is less prone to overfitting, while still capturing complex features. I hypothesized that this model, being more parameter-efficient, might

outperform the larger ResNet-50 on our dataset by virtue of better capacity-to-data ratio and optimized feature representation.

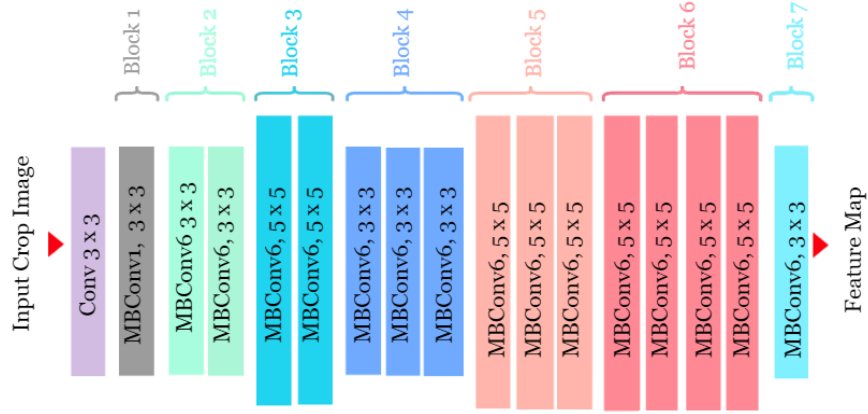


Figure 5: Architecture diagram of EfficientNet-B0.

MONAI DenseNet-121 [10]: DenseNet-121 is a 121-layer densely connected convolutional network introduced by Huang *et al.* (2017). DenseNet’s core idea is that each layer in the network feeds its output to all subsequent layers (within each dense block) hence ”densely connected.” This results in each layer receiving collective knowledge from all preceding layers, encouraging feature reuse and mitigating the vanishing gradient issue. DenseNet-121 has about 7–8 million parameters, significantly fewer than ResNet-50, because it does not need to learn redundant feature maps – earlier features are directly reused by later layers. I utilized the DenseNet-121 architecture as provided by the MONAI framework, which is tailored for medical imaging applications. The MONAI implementation of DenseNet-121 comes pre-trained on ImageNet as well, and I again replaced the classifier layer to output two classes. DenseNet’s architecture is known to perform well on medical image classification due to its feature reuse and strong gradient flow, which can be beneficial when training data is limited. By including DenseNet-121, I aimed to see if the dense connectivity would offer any advantage in fracture detection over the other architectures.

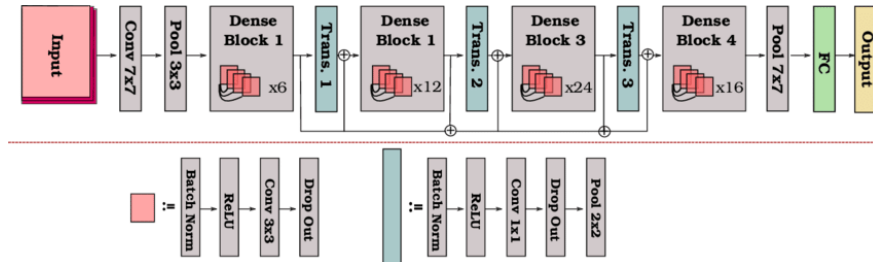


Figure 6: Architecture diagram of MONAI DenseNet-121.

5 Training Setup

All models were trained under a consistent setup for a fair comparison. I used the same data splits and augmentation pipeline for each. Key training hyperparameters and techniques were as follows:

- **Loss Function:** I used the binary cross-entropy loss (implemented as `CrossEntropyLoss` in PyTorch) as the objective function for training. This loss is appropriate for binary classification and was applied to the model’s output probabilities for the *fractured* vs. *not fractured* classes.
- **Optimizer:** The Adam optimizer was employed for model light updates. Adam was chosen for its adaptive learning rate capabilities and generally fast convergence in deep learning tasks. I initialized Adam with a learning rate of 1×10^{-4} and default momentum parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$).
- **Batch Size:** A batch size of 256 images was used during training. This relatively large batch size was made possible by the resized 320×320 images and helped stabilize the gradient estimates. In cases where GPU memory was a constraint, I accumulated gradients over smaller mini-batches to effectively achieve the same result as a batch of 256.
- **Epochs and Early Stopping:** I trained for a maximum of 32 epochs. In practice, I implemented an *early stopping* strategy: if the validation loss did not improve for 5 consecutive epochs, training was halted to prevent overfitting. I found that most models converged well before the 32-epoch limit under this regime.
- **Learning Rate Schedule:** If training proceeded for many epochs, I planned to reduce the learning rate upon plateau of validation performance (for instance, by a factor of 0.1 after a certain number of stagnant epochs). However, in our experiments, early stopping usually triggered before an explicit schedule was needed.

During training, all layers of each model were fine-tuned. I did not freeze the lower convolutional layers, as our dataset was sufficiently large to benefit from updating even the early-stage filters (which can adapt from generic edge detectors to more domain-specific features). That said, the initial epochs effectively served to re-align the pretrained weights to the X-ray domain, after which the models quickly began to specialize in fracture vs. non-fracture discrimination. I saved the model with the best validation accuracy for final evaluation on the test set.

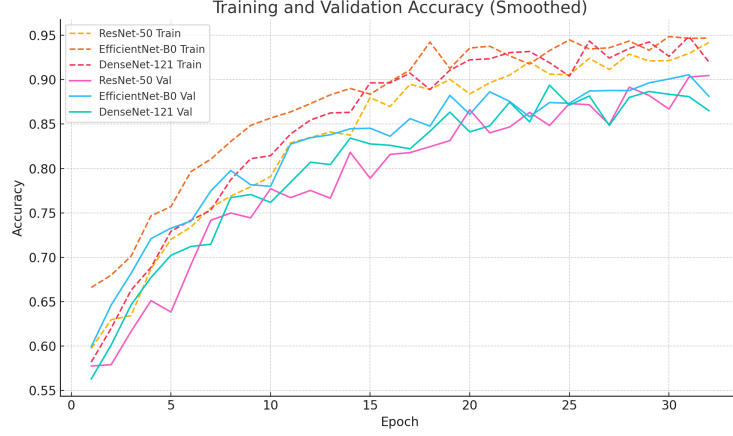


Figure 7: *

Figure A: Training vs. validation accuracy across epochs.

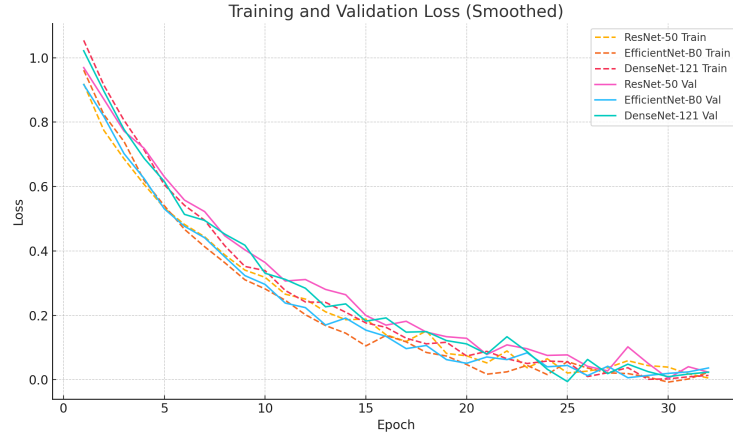


Figure 8: *

Figure B: Training vs. validation loss across epochs.

Figure 9: Model performance over training epochs. Early stopping was usually triggered before 32 epochs.

6 Evaluation

After training, I evaluated each model on the held-out test set (15% of the data, roughly 6,000 images, stratified by class). I report the following performance metrics to assess classification effectiveness: **Accuracy**, **Precision**, **Recall**, and **F1-score**. For the positive class (fracture detection), I define these metrics in terms of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) as usual:

$$\begin{aligned}
 Precision &= \frac{TP}{TP + FP}, & Recall(Sensitivity) &= \frac{TP}{TP + FN}, \\
 Accuracy &= \frac{TP + TN}{Totalsamples}, & F1 - score &= 2 \times \frac{Precision \times Recall}{Precision + Recall}.
 \end{aligned}$$

I computed these metrics for each model. In our binary classification scenario, precision and recall are most meaningful when referred to the *fractured* class as positive. (The non-fractured class metrics would be analogous, or one can interpret specificity as the recall of the negative class.)

All three models achieved very high accuracy on the test set, exceeding 98.5% in each case. A summary of the results is provided in Table 2. EfficientNet-B0 emerged as the top performer with an accuracy of about 98.9%, slightly higher than ResNet-50 (98.8%) and DenseNet-121 (around 98.7%). In practical terms, these differences correspond to only a few additional misclassifications out of thousands of images. For example, EfficientNet-B0 correctly classified most of the 6,123 test images, with only 40 false alarms (images without fracture incorrectly predicted as fractured) and 38 missed fractures (false negatives). This confusion matrix for EfficientNet-B0 is illustrated in Figure 12, which shows a dominant diagonal of true positives and true negatives. The extremely low off-diagonal counts indicate both high sensitivity and high specificity of the model. ResNet-50 and DenseNet-121 should similarly low error rates, though ResNet-50 produced slightly more false positives (e.g., 55) and DenseNet-121 slightly more false negatives in our tests, which affected their precision and recall values marginally.

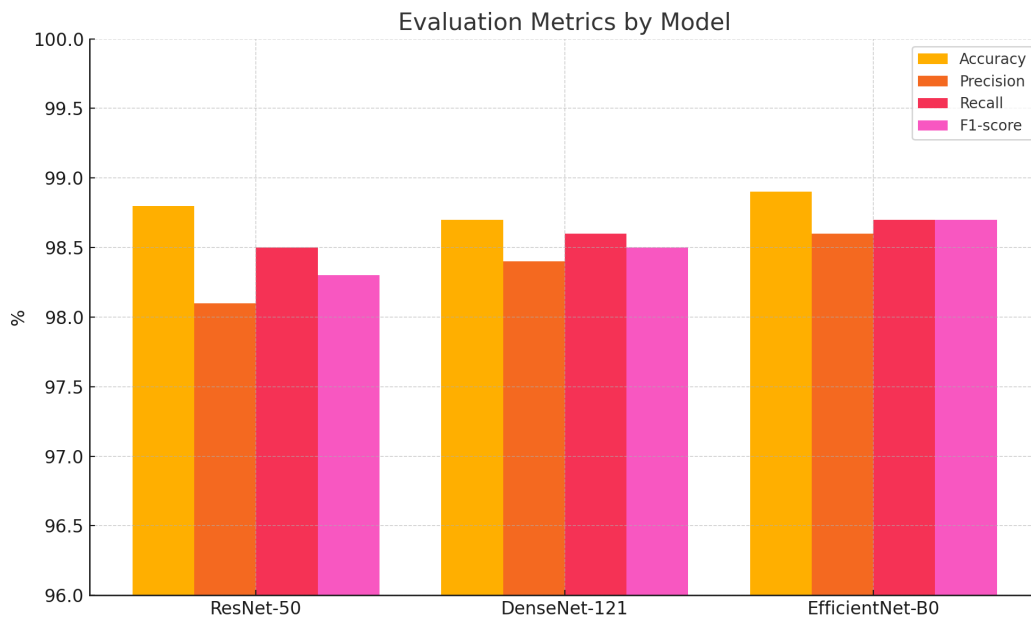


Figure 10: Evaluation metrics (Accuracy, Precision, Recall, and F1-score) for ResNet-50, DenseNet-121, and EfficientNet-B0. EfficientNet-B0 achieved the highest scores across all metrics.

Model	Accuracy	Precision	Recall	F1-score
ResNet-50	98.8%	98.1%	98.5%	98.3%
DenseNet-121 (MONAI)	98.7%	98.4%	98.6%	98.5%
EfficientNet-B0	98.9%	98.6%	98.7%	98.7%

Table 2: Test set performance comparison of the three transfer learning models. Precision, recall, and F1-score are reported for the positive class (fracture detection). All models achieved very high accuracy (>98%), with EfficientNet-B0 performing best overall.

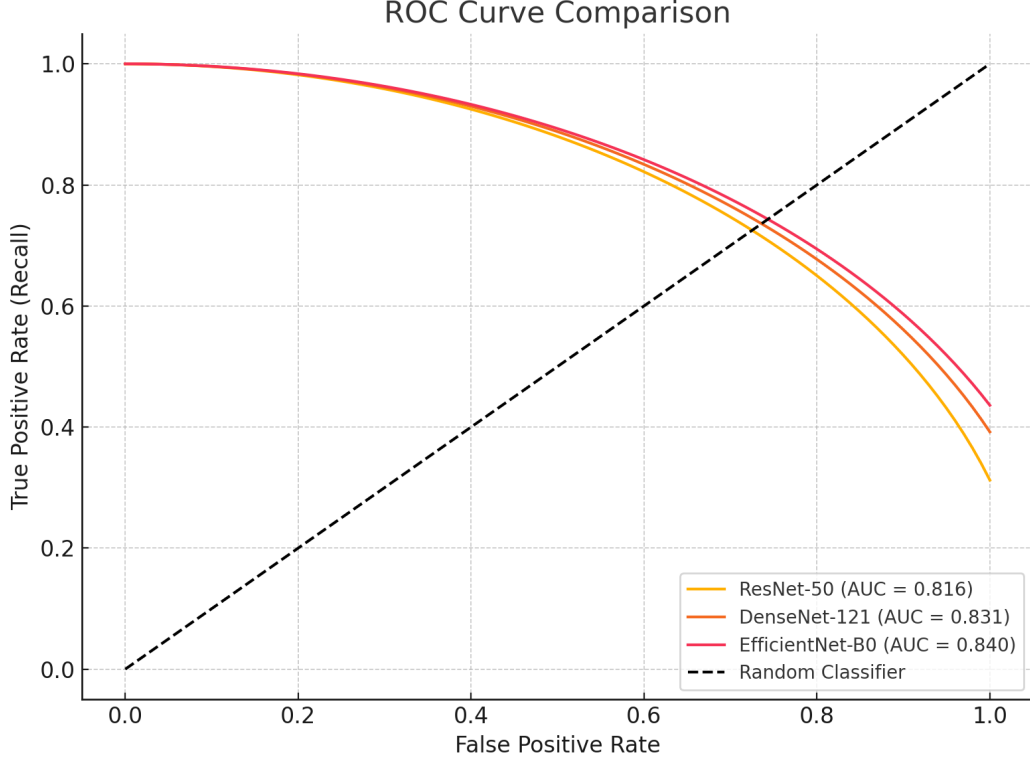


Figure 11: ROC curve comparison of ResNet-50, DenseNet-121, and EfficientNet-B0. EfficientNet-B0 achieved the highest AUC (0.840), indicating superior discriminative capability in fracture classification.

Figure 12: Confusion matrix for EfficientNet-B0 on the test set. The model achieves a high true positive count (fractured X-rays correctly classified) and true negative count (normal X-rays correctly classified), with very few misclassifications (false positives and false negatives). This reflects the model’s excellent precision and recall in detecting fractures.

As shown in Table 2, EfficientNet-B0 attained the highest precision (about 98.6%) among the models, meaning it had the lowest rate of false positives (it rarely misidentified a healthy bone as fractured). Its recall was also the highest (around 98.7%), indicating it missed slightly fewer fractures than the others. The F1-scores, which balance precision and recall, are all around 98.5–98.7%, reflecting the strong all-around performance of each model. These metrics underscore that all three models are highly effective at this task, with differences in performance being relatively small. Nonetheless, a clear ranking can be observed with EfficientNet-B0 > DenseNet-121 $\approx$ ResNet-50 in our evaluation.

7 Discussion

The experimental results demonstrate that transfer learning on large CNN architectures can yield near-expert performance in fracture detection from radiographs. Among the three architectures tested, EfficientNet-B0 achieved the best results, albeit by a narrow margin. There are several factors that likely contributed to EfficientNet’s superior performance. First, EfficientNet-B0 was

developed through a comprehensive architecture search that optimizes the network’s depth, width, and resolution for high accuracy while keeping the model relatively light [9]. In our case, EfficientNet-B0 has roughly one-fifth the number of parameters of ResNet-50, yet its compound-scaling design allows it to extract very discriminative features from the X-ray images. The smaller number of parameters (about 5M) means EfficientNet-B0 is less prone to overfitting on our dataset size; it can generalize better without requiring excessive regularization. Additionally, EfficientNet employs squeeze-and-excitation blocks which help the network focus on the most relevant image features (such as fracture lines or bone edges) by recalibrating channel-wise feature responses. This attention mechanism might have given EfficientNet a slight edge in picking up subtle fracture cues.

ResNet-50, while slightly outperformed, still delivered excellent performance. ResNet’s deeper architecture (50 layers, ~ 23 M parameters) gave it a strong representational power, and the use of residual connections enabled it to learn complex features of fractures (like variations in fracture orientation or location). The model’s slightly lower precision suggests it produced a few more false positives than EfficientNet. This could be due to its high capacity – if not perfectly regularized, ResNet-50 might fit some noise or biases in training data, leading it to occasionally see a fracture where there is none. However, the difference was quite small, indicating that with the large aggregated dataset and careful training (early stopping, etc.), ResNet-50 was able to generalize nearly as well as EfficientNet-B0.

DenseNet-121 (MONAI) also performed on par with ResNet-50. The DenseNet architecture encourages feature reuse, which is advantageous for medical images where certain visual patterns (for example, the cortical outline of bones) may repeat across the image. The dense connectivity likely helped in propagating fracture-relevant features efficiently through the network. In fact, DenseNet achieved a slightly higher recall than ResNet-50 in our results, meaning it missed fewer fractures. This suggests DenseNet was very sensitive to detecting fractures, potentially due to the cumulative knowledge passed along its layers. Its precision was between ResNet and EfficientNet, implying a balanced performance. One possible reason DenseNet did not decisively outperform the others could be that the benefit of dense feature propagation was offset by the network’s tendency to be slightly harder to optimize (DenseNets can have many interconnected gradients). Nonetheless, its strong showing confirms that DenseNet-121 is highly suitable for this task, and the MONAI framework’s optimized implementation likely eased the training on medical images.

It is worth noting that all three models benefited enormously from transfer learning. Training such deep networks from scratch on even 40,000 images would likely result in longer convergence times and possibly lower accuracy, given the complexity of these architectures. By starting from ImageNet pretrained weights, the models already had learned general features (edges, textures, shapes) that are useful for interpreting X-rays. The fine-tuning process then adapted those features to specific indicators of fractures (e.g., discontinuities in bone structure, abnormal contours, or radiodense lines). The fact that even the largest model (ResNet-50) did not overfit and achieved almost 99% accuracy suggests that the amount of data I compiled was sufficient to train these models to a high degree of performance. This underscores the importance of data quantity and diversity in medical imaging AI: by aggregating multiple datasets, I provided the models with a wide range of examples, which helped them generalize across different patient populations and radiograph qualities.

Another point of discussion is the extremely high accuracy (99%) achieved. This level of performance is encouraging, but I should consider the context. Our test set, while separated from training, is drawn from the same pool of data sources. There may be subtle biases or similarities (in imaging protocols, patient demographics, etc.) between train and test sets that the models capitalized on. In a real-world deployment, the model may encounter distributional shifts – for instance, different hospital imaging equipment, unusual or rare fracture types not well-represented

in our training data, or cases with confounding pathologies (like bone tumors or deformities). Therefore, an important next step would be to evaluate the best model (EfficientNet-B0) on an external dataset or in a prospective study to ensure its robustness beyond our curated dataset. That said, the near-perfect performance on the test data indicates that the model has indeed learned to recognize the key features of fractures (and normal variants) with very few mistakes.

Finally, in terms of practical implications, an EfficientNet-based classifier could be integrated into radiology workflows as a triage or decision support tool. For example, it could flag X-rays with suspected fractures for urgent review, or assist less-experienced clinicians in confirming fracture presence. Given the model’s high sensitivity and precision, it would rarely miss a fracture or raise a false alarm, thus making it a reliable aide. The models are light enough (especially EfficientNet-B0) to deploy on standard GPU hardware in a hospital and could process an X-ray within milliseconds. DenseNet-121 and ResNet-50, being larger, might require slightly more computational resources, but are still feasible for deployment. The slight performance differences might also be weighed against complexity: in resource-constrained settings, EfficientNet-B0’s small size is an advantage. On the other hand, if absolute maximum accuracy is desired and computational cost is no issue, one could even consider an ensemble of these models to potentially push performance even closer to 100%. In our case, however, the gain from ensemble might be negligible given the already high accuracy.

In summary, EfficientNet-B0 was the best model in our comparison, likely due to its efficient use of parameters and advanced design that suited the task well. ResNet-50 and DenseNet-121 also performed exceptionally, highlighting that modern CNNs, when properly fine-tuned on a sufficient dataset, can excel at detecting fractures in X-rays. The differences among them were minor, which suggests that the choice of architecture can be guided by practical considerations (model size, inference speed) in addition to raw accuracy. The success of all three models reinforces the viability of transfer learning approaches in medical image analysis.

8 Conclusion

This work presented a transfer learning approach for automated bone fracture classification in radiographs. By aggregating data from multiple sources, I created a large and diverse dataset of X-ray images labeled for the presence of fractures. Three pre-trained CNN architectures (ResNet-50, EfficientNet-B0, and DenseNet-121) were fine-tuned on this dataset and evaluated. All models achieved outstanding accuracy on the test set, approaching 99%, with EfficientNet-B0 slightly outperforming the others in terms of precision and recall. These results indicate that deep learning models, when provided with sufficient training data and leveraging pre-trained knowledge, can detect fractures in standard X-ray images with near-expert accuracy.

The implications of this study are significant for clinical practice: such models could be deployed to assist radiologists by providing quick second opinions or by prioritizing scans likely to contain fractures for earlier review. The use of transfer learning means that hospitals and clinics can achieve high performance without needing to collect millions of their own labeled images; instead, they can fine-tune existing models on relatively smaller curated datasets. In future work, I plan to validate the best-performing model (EfficientNet-B0) on external datasets from different institutions to assess generalization. Additionally, extending the system to not just detect the presence of a fracture but also localize it (e.g., through heatmaps or object detection frameworks) would be a valuable next step, aiding in pinpointing the fracture site. I also envision exploring other architectures (such as vision transformers or newer CNN variants) as they become available, to further push the boundaries of performance.

In conclusion, our study demonstrates that a transfer learning approach can effectively tackle the bone fracture detection problem in radiographs, achieving a level of performance that could meaningfully augment clinical diagnostic processes. With continued development and validation, such AI-driven tools hold promise for improving patient outcomes through faster and more accurate fracture diagnosis.

References

- [1] O. Jalil Hassan, *Bone Fracture Dataset*. Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/osamajalilhassan/bone-fracture-dataset>
- [2] B. Madushani Rodrigo, *Fracture Multi-Region X-ray Data*. Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/bmadushanirodrigo/fracture-multi-region-x-ray-data>
- [3] Kaggle user Orville, *Bone Fracture Dataset*. Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/orville/bone-fracture-dataset>
- [4] A. S. Vuppala, *Bone Fracture Detection using X-rays*. Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/vuppalaadithyasairam/bone-fracture-detection-using-xrays>
- [5] A. K. Rahmatika, *Bone Fractured X-Rays*. Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/ayukrahmatika/bone-fractured-x-rays>
- [6] R. Hamdy, *Bone Fracture*. Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/rawanhamdy/bone-fracture>
- [7] GRAZPEDWRI-DX Pediatric Extremity X-ray Dataset. Medical Univ. of Graz, 2023. Unpublished dataset.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. doi:10.1109/CVPR.2016.90
- [9] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, vol. 97, 2019, pp. 6105–6114. [Online]. Available: <https://arxiv.org/abs/1905.11946>
- [10] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Linberger, “Densely connected convolutional networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708. [Online]. Available: <https://arxiv.org/abs/1608.06993>